



READING SHEET

API Routes & Route Handlers

Module 3 · Frontend: React & Next.js · Day 39 · Week 8

PROGRAM

CodeOps · Full Stack

WORK TYPE

Revision + mini-project

EST. TIME

2–3 hours

Tip How to use this sheet

This is your take-away for Day 39. Your application can already render on the server; today it learns to answer requests as well. Route handlers expose real HTTP endpoints, and server actions are the one kind of function that may cross the boundary you met yesterday. By the end both halves of Addis Eats live in one repository — and the imaginary server your Day 33 checkout was talking to becomes real.

Part 1 · Route Handlers

A `route.js` file makes a folder answer HTTP requests instead of rendering a page. The same folder-as-URL rule applies, and brackets still make a segment dynamic — only the file name and the return value change.

```
app/api/dishes/route.js -> GET /api/dishes

export async function GET() {
  const dishes = await db.dish.findMany();
  return Response.json(dishes);
}

// one export per method: GET POST PUT PATCH DELETE
```

Warning `page.js` or `route.js`, never both

A folder containing both will not build. Keep endpoints under `app/api/` and pages everywhere else, and the question never arises.

Request and Response

These are the same Request and Response objects the Fetch API uses — the ones you called from the browser on Day 29. Learning them here means learning them everywhere.

```
web standards, not a framework API

export async function POST(request) {
  const body = await request.json();
  const area = request.headers.get("x-area");
  return Response.json({ ok: true }, { status: 201 });
}

// request.json() request.formData() request.text()
```

Params and query strings

```
app/api/dishes/[id]/route.js

export async function GET(request, { params }) {
  const dish = await getDish(params.id);
  if (!dish) return Response.json(
    { error: "No such dish" }, { status: 404 });
  return Response.json(dish);
}

// query string comes from the request, not the route:
new URL(request.url).searchParams.get("category")
```

A path parameter arrives in the second argument, exactly as on a page. The query string comes from the request URL because it belongs to the request rather than the route. Both are strings, and both are user input — parse and check them before use.

Caching applies here too

IN A HANDLER	EFFECT
A plain GET	May be cached like a static route
Reading request.url, cookies or headers	Forces it dynamic
revalidate = 60	Cached, refreshed at most once a minute
Any POST, PUT or DELETE	Never cached

Route handlers obey the same Day 37 rules as pages and appear in the same build output. If a GET endpoint returns stale data, check first whether it was prerendered at build time — that is usually the answer.

Part 2 · Building an Endpoint

An endpoint is a contract: a method, a shape going in, a shape coming back, and a status code that tells the caller what happened. Getting those four right is most of what makes an API usable.

CODE	WHEN TO RETURN IT
200	Here is what you asked for
201	Created — return the new resource
400	The request itself was malformed
401	You are not signed in
403	Signed in, but not allowed
404	No such thing
422	Well-formed, but the values failed validation
500	We broke, not you

Your Day 33 checkout branched on 422 to show field errors — that only works because the server chose the right number. Returning 200 with an error buried inside forces every client to guess. Note the distinction: **401** means signing in would fix it; **403** means it would not.

Validate before you trust

```
app/api/orders/route.js

export async function POST(request) {
  const body = await request.json();
  const result = orderSchema.safeParse(body);

  if (!result.success) return Response.json(
    { fieldErrors: result.error.flatten().fieldErrors },
    { status: 422 });

  const order = await createOrder(result.data);
  return Response.json(order, { status: 201 });
}
```

Warning Anyone can post to this

The zod schema from Day 33 validated the form in the browser; here it validates the request body. One file, two enforcement points — and the client version is only a courtesy. A student can send a request with curl and no form at all, so every rule that matters (the TeleBirr pattern, the price, the delivery area) is enforced here or not at all.

A predictable error shape

```
decide the shape once, use it everywhere

// success
{ "id": "ord_812", "total": 640, "currency": "ETB" }

// failure — always the same keys
{ "error": "Validation failed",
  "fieldErrors": { "phone": ["Use 09... or +2519..."] } }
```

When every endpoint fails the same way, the calling code has one error path instead of one per endpoint — that consistency is worth more than any individually clever response. Log the stack trace on the server and send the caller a message they can act on: a database error in the response body is an invitation.

Part 3 · Server Actions

Yesterday you learned that functions cannot cross the boundary. A **server action** is the exception: a function marked "use server" that a client component may call directly, with the network request generated for you. You never write a URL, a method or a JSON.stringify.

```
app/actions.js

"use server";

export async function placeOrder(formData) {
  const parsed = orderSchema.safeParse({
    name: formData.get("name"), phone: formData.get("phone") });
  if (!parsed.success) return { fieldErrors: /* ... */ };
  return { id: (await createOrder(parsed.data)).id };
}
```

The directive mirrors the one from yesterday: "use client" marks code for the browser, "use server" marks a function that must only ever run on the server, however it is called.

Forms that work before hydration

```
import { placeOrder } from "../actions";

<form action={placeOrder}>
  <input name="name" />
  <input name="phone" />
  <button>Order</button>
</form>
```

the action attribute takes a function

This form submits even if the JavaScript has not loaded, because it falls back to a normal browser submission — on a weak connection in Addis that is a real difference. The page works without JavaScript and works better with it, which was hard to achieve with the Day 33 approach and is the default here.

Pending state and revalidation

```
"use client";
const [state, formAction, pending] =
  useActionState(placeOrder, null);

<button disabled={pending}>
  {pending ? "Sending..." : "Order"}</button>

// inside the action, after a successful write:
revalidatePath("/orders");
```

useActionState, and refreshing what changed

Day 33's submitting, failed and succeeded states came from state you managed yourself; here the hook returns the result and the pending flag. `revalidatePath` and `revalidateTag` tell Next.js which cached pages are now wrong — without them a new order will not appear on a statically generated list.

An action is a public endpoint

```
"use server";
export async function cancelOrder(orderId) {
  const user = await getSession();
  if (!user) throw new Error("Not signed in");
  const order = await getOrder(orderId);
  if (order.userId !== user.id) throw new Error("Not yours");
  await markCancelled(orderId);
}
```

check inside the action, every time

Warning Hiding the button is not security

Every server action becomes a real HTTP endpoint that anyone can call with any argument. The check must live inside the action, not in the component that renders the button. The most common mistake is trusting an id passed from the client — always confirm the signed-in person is allowed to touch that particular record.

Part 4 · Choosing an Approach

You now have three ways to reach the server. They are not interchangeable, and picking the wrong one shows up as either extra code or a security hole.

USE	WHEN	EXAMPLE
Server component	Reading data to render a page	The menu, a dish, a receipt
Server action	Writing data from your own UI	Placing or cancelling an order
Route handler	Something outside your UI calls it	A webhook, a mobile app
Client fetch	The person triggers it repeatedly	Search as you type, polling

If only your own pages need it, prefer a server component to read and a server action to write — there is no URL to design, no fetch to write and nothing to keep in sync. Reach for a route handler the moment something you do not control needs to call it: another application, a payment provider posting back, or a client that is not this website.

Where the checks belong

NOT SECURITY	SECURITY
Hiding a button	Checking the session in the action
Disabling an input	Validating the body on the server
A guarded route	Confirming the record belongs to them
Client-side validation	Re-validating with the same schema
An id the client sent	An id you looked up from the session

Everything on the left is user experience — it makes the interface pleasant and coherent, and none of it stops a request being sent by hand. Day 31's `RequireAuth` stopped someone browsing to the checkout; it never stopped anyone posting an order.

Keeping secrets on the server

```

DATABASE_URL=postgres://...      # server only
TELEBIRR_SECRET=sk_live_...      # server only
NEXT_PUBLIC_MAP_KEY=pk_...        # sent to the browser
.env.local
```

The prefix is the switch. Without `NEXT_PUBLIC` a variable simply does not exist in client code; with it, the value is baked into the bundle and readable by anyone — so use it only for keys meant to be public. Check that `.env.local` is in `.gitignore` before your first push: a leaked key in a public repository is found by automated scanners within minutes.

Assigned Reading

Required

1. [nextjs.org — Route Handlers](#) What / why: `route.js`, the supported methods, caching behaviour and dynamic segments.
2. [nextjs.org — Server Actions and Mutations](#) What / why: the directive, forms, pending state, revalidation and the security section.

Optional — go deeper

- [Environment variables](#) — what `NEXT_PUBLIC` does, and what stays on the server.

- [HTTP response status codes](#) — the full list, with the ones you will actually use explained.

Exercises

Test each endpoint with curl before wiring any UI to it.

1. Build GET /api/dishes returning the full menu as JSON.
2. Add GET /api/dishes/[id], returning 404 with the standard error shape for an unknown id.
3. Add POST /api/orders validating the body with your Day 33 schema, returning 422 with fieldErrors.
4. Point the checkout form at it and branch on the status code exactly as you did on Day 33.
5. Rewrite the same submission as a server action and delete the fetch call entirely.
6. Add useActionState for the pending flag, and revalidatePath after a successful write.
7. Write a cancelOrder action that checks the session and the record owner before writing.

Mini-Project · Addis Eats Gets an API

Tip What you build

The server half of the application, in the same repository as the pages. A dishes API with a dynamic segment, an orders endpoint validating with the schema your form already uses, the same submission rewritten as a server action with pending state and cache revalidation, and an authorisation check that cannot be bypassed by hiding a button.

Requirements

- GET /api/dishes and GET /api/dishes/[id], with an honest 404 and one error shape.
- POST /api/orders returning 201 on success and 422 with fieldErrors on invalid input.
- A placeOrder server action replacing the fetch, with useActionState driving the button.
- revalidatePath called after a successful write so cached pages are refreshed.
- A cancelOrder action verifying both the session and the ownership of the record.
- .env.local in .gitignore, with no secret readable from client code.

What to submit

Your GitHub repository with the handlers, the actions and the shared schema, plus a README listing each endpoint, its method and its status codes.

Check yourself

- Does curl -X POST with an invalid phone number return 422 and named field errors?
- Does a valid order return 201, and an unknown dish id 404 rather than an empty 200?
- Does the form still show field errors after you switched to the server action?
- Does a new order appear immediately on a cached orders page?
- Calling cancelOrder with someone else's id from the console — what happens?
- Can you find any secret value in the JavaScript the browser downloaded?